JavaScript

Destructuring

A Complete Beginner-Friendly Guide

Core JavaScript Concepts · Easy English · With Examples

1. What is Destructuring?

Destructuring is a way to **take values out of an array or object** and put them into separate variables — all in one short line of code.

Without destructuring, you would write:

```
const fruits = ['Apple', 'Banana', 'Mango'];

const first  = fruits[0];

const second = fruits[1];

const third  = fruits[2];
```

With destructuring, you write just one line:

```
const [first, second, third] = fruits;
```

■ *Same result, but much shorter and cleaner!*

2. Array Destructuring

2.1 Basic Array Destructuring

Use square brackets **[ ]** on the left side to grab items from an array.

```
const colors = ['Red', 'Green', 'Blue'];


const [a, b, c] = colors;


console.log(a);  // Red

console.log(b);  // Green

console.log(c);  // Blue
```

2.2 Skip Items with Commas

Leave a gap (just a comma) to skip items you don't need.

```
const nums = [10, 20, 30, 40];


const [, second, , fourth] = nums;
```

```
console.log(second);  // 20

console.log(fourth);  // 40
```

2.3 Default Values

If an item doesn't exist in the array, you can give it a fallback value.

```
const [x = 5, y = 10] = [100];


console.log(x);  // 100  (taken from array)

console.log(y);  // 10   (array had no second item, used default)
```

2.4 Swap Two Variables

Destructuring makes swapping variables very easy — no temp variable needed!

```
let p = 'Hello';

let q = 'World';


[p, q] = [q, p];


console.log(p);  // World

console.log(q);  // Hello
```

2.5 Rest Operator (...rest)

Use **...rest** to collect all remaining items into a new array. The rest element must always be the **last** in the list.

```
const [head, ...tail] = [1, 2, 3, 4, 5];


console.log(head);  // 1

console.log(tail);  // [2, 3, 4, 5]
```

3. Object Destructuring

3.1 Basic Object Destructuring

Use curly braces **{ }** on the left side. The variable names must match the object's property names.

```
const person = { name: 'Arjun', age: 25, city: 'Mumbai' };


const { name, age, city } = person;
```

```
console.log(name);  // Arjun

console.log(age);   // 25

console.log(city);  // Mumbai
```

3.2 Rename While Destructuring

You can give the extracted value a **new variable name** using a colon **:**.

```
const user = { username: 'john_doe', score: 99 };


const { username: playerName, score: points } = user;


console.log(playerName);  // john_doe

console.log(points);      // 99
```

3.3 Default Values in Objects

If a property doesn't exist in the object, the default value is used.

```
const config = { host: 'localhost' };


const { host, port = 3000 } = config;


console.log(host);  // localhost

console.log(port);  // 3000  (default, not in object)
```

3.4 Rest in Objects

Grab some properties, and collect the remaining ones into a new object.

```
const product = { id: 1, name: 'Shoe', price: 500, stock: 20 };


const { id, name, ...details } = product;


console.log(id);       // 1

console.log(name);     // Shoe

console.log(details);  // { price: 500, stock: 20 }
```

4. Nested Destructuring

When objects or arrays are inside other objects/arrays, you can go deeper.

4.1 Nested Object

```
const student = {
  name: 'Priya',
  address: {
    city: 'Delhi',
    pin: '110001'
  }
};

const { name, address: { city, pin } } = student;

console.log(name);  // Priya
console.log(city);  // Delhi
console.log(pin);   // 110001
```

4.2 Nested Array

```
const matrix = [[1, 2], [3, 4]];

const [[a, b], [c, d]] = matrix;

console.log(a, b, c, d);  // 1 2 3 4
```

5. Destructuring in Function Parameters

Instead of passing a full object and then accessing its properties, you can destructure right inside the function definition.

5.1 Object Parameter Destructuring

```
// Without destructuring
function greet(user) {
  console.log('Hello, ' + user.name + '! Age: ' + user.age);
}

// With destructuring
function greet({ name, age }) {
  console.log(`Hello, ${name}! Age: ${age}`);
```

```
}

greet({ name: 'Riya', age: 22 });  // Hello, Riya! Age: 22
```

5.2 Array Parameter Destructuring

```
function getFirst([first, second]) {
  return `First: ${first}, Second: ${second}`;
}

console.log(getFirst(['Sun', 'Mon']));  // First: Sun, Second: Mon
```

5.3 Default Values in Parameters

```
function createUser({ name = 'Guest', role = 'viewer' } = {}) {
  console.log(`${name} is a ${role}`);
}

createUser({ name: 'Admin', role: 'editor' });  // Admin is a editor
createUser({});                                  // Guest is a viewer
createUser();                                    // Guest is a viewer
```

■ *The = {} at the end is important — it prevents an error if no argument is passed.*

6. Real-World Examples

6.1 API Response Handling

When you get data from an API, destructuring helps pick what you need quickly.

```
const apiResponse = {
  status: 200,
  data: {
    userId: 42,
    posts: ['Post A', 'Post B']
  },
  error: null
};
```

```
const { status, data: { userId, posts } } = apiResponse;


console.log(status);  // 200

console.log(userId);  // 42

console.log(posts);   // ['Post A', 'Post B']
```

6.2 Import Only What You Need

ES6 module imports use object destructuring under the hood.

```
// Instead of importing everything:

import React from 'react';


// You can pick just what you need:

import { useState, useEffect } from 'react';
```

6.3 Returning Multiple Values from a Function

A function can return an array or object, and you can destructure the result.

```
function getMinMax(arr) {

  return { min: Math.min(...arr), max: Math.max(...arr) };

}


const { min, max } = getMinMax([3, 1, 7, 2, 9]);


console.log(min);  // 1

console.log(max);  // 9
```

6.4 Loop over Array of Objects

```
const users = [

  { name: 'Alice', age: 30 },

  { name: 'Bob',   age: 25 },

];


for (const { name, age } of users) {

  console.log(`${name} is ${age} years old`);

}
```

```
// Alice is 30 years old

// Bob is 25 years old
```

7. Quick Reference Table

Syntax	What It Does	Example
`[a, b] = arr`	Array destructuring	`[x, y] = [1, 2]`
`[a, , c] = arr`	Skip an element	`[, second] = [10, 20]`
`[a = 5] = arr`	Default for array	`[x = 0] = []`
`[a, ...rest] = arr`	Rest of array	`[h, ...t] = [1,2,3]`
`{ a, b } = obj`	Object destructuring	`{ name } = person`
`{ a: newName }`	Rename property	`{ name: n } = user`
`{ a = val }`	Default for object	`{ port = 80 } = cfg`
`{ a, ...rest }`	Rest of object	`{ id, ...more } = obj`
`{ a: { b } }`	Nested object	`{ addr: { city } }`

8. Common Mistakes to Avoid

- **Wrong variable name**: In object destructuring, variable name must match the key exactly (unless you rename it with `:`).
- **Forgetting = {}**: When destructuring a function parameter that might be undefined, always add `= {}` as a default.
- **Rest not last**: The rest element (`...rest`) must always be the last item — you cannot have anything after it.
- **Confusing [ ] and { }**: Use `[ ]` for arrays and `{ }` for objects. Mixing them up causes errors.
- **Deeply nested can get messy**: More than 2 levels of nesting is hard to read. Consider using intermediate variables instead.

9. Pro Tips

- Use destructuring in **for...of** loops to make iteration over object arrays clean.
- Combine destructuring with the **spread operator** (`...`) for powerful data manipulation.
- Destructuring works great with **Promise.all**: `const [res1, res2] = await Promise.all([...]);`
- You can mix array and object destructuring: `const [{ name }] = users;`
- Destructuring is pure **ES6** — it works in all modern browsers and Node.js without any library.

End of Document — Happy Coding! ■